

Reversing a String

Like summing the numbers in an array, reversing a string is another frequently cited recursive algorithm even though the iterative solution is straightforward. Because a string is essentially an array of single characters, we'll employ the head and tail approach for our `rev()` function just as we did for the summation algorithm.

Let's start with the smallest strings possible. A blank string and a single-character string are already the reverse of themselves. These naturally form our base cases: if the string argument is a string such as `' '` or `'A'`, our function should simply return the string argument.

For larger strings, let's try splitting the string into a head (just the first character) and tail (all characters after the first). For a two-character string like `'XY'`, `'X'` is the head and `'Y'` is the tail. To reverse the string, we need to place the head behind the tail: `'YX'`.

Does this algorithm hold for longer strings? To reverse a string like `'CAT'`, we would break it into the head `'C'` and the tail `'AT'`. But placing the head behind the tail alone doesn't reverse the string; it gives us `'ATC'`. What we actually want to do is put the head behind *the reverse of* the tail. In other words, `'AT'` would reverse to `'TA'`, and then adding the head to the end of that would produce the reversed string, `'TAC'`.

How can we reverse the tail? Well, we can recursively call `rev()` and pass it the tail. Forget about the implementation of our function for a moment and focus on its input and output: `rev()` takes one string argument and returns a string with the argument's characters reversed.

Thinking about how to implement a recursive function like `rev()` can be difficult because it involves a chicken-and-egg problem. In order to write `rev()`'s recursive case, we need to call a function that reverses a string—that is, `rev()`. As long as we have a solid understanding of what our recursive function's arguments and return value will be, we can use the *leap-of-faith* technique to get around this chicken-and-egg problem by writing our recursive case assuming the `rev()` function call returns the correct value even though we haven't finished writing it yet.